PRD: Agent Recorder as an MCP Server

Feature: Expose Agent Recorder's observability data and event ingestion as a standard MCP server
Author: Edyta Kucharska
Status: Draft
Version: 1.0
Date: 2026-03-26
Package: `packages/mcp-server` (new)

---

1. Problem Statement

Agent Recorder captures a rich, structured timeline of agent execution — tool calls, subagent delegations, skill invocations, token consumption, and error states — all stored locally in SQLite with parent/child relationships preserved. Today, this data is accessible only through the AR CLI and the local web UI.

This creates three problems:

Problem 1: Observability data is trapped inside AR. External agent runtimes (n8n, LangGraph, CrewAI, Syrin, Make.com workflows) cannot query what happened in a Claude Code session. A developer running a mixed workflow — Claude Code for coding, n8n for orchestration — has no unified timeline. The data exists but there's no standard interface to reach it.

Problem 2: Non-Claude-Code agents can't record to AR. Agent Recorder currently only captures events from MCP traffic it proxies. An n8n workflow, a Python script using LangGraph, or a Syrin agent cannot report their own execution events into AR's timeline. There is no observability tool that works across agent runtimes using a standard protocol.

Problem 3: Token and cost roll-up data has no programmatic consumer. Since v2.0.14, AR captures token counts per tool call and monitors context budget. But this data can only be viewed through the CLI or UI. An agent that wants to check its own token budget mid-session, or a monitoring dashboard that wants to aggregate costs across sessions, has no API to call.

Why MCP?

MCP is the natural interface because:

- AR already speaks MCP — it's an MCP proxy. Adding an MCP server is architecturally consistent.
- Any MCP client (Claude Code, Cursor, VS Code Copilot, OpenAI Agents SDK, n8n, custom agents) can connect without custom integration code.
- The protocol handles tool discovery, typed parameters, and error reporting out of the box.
- It aligns with the project principle "local-first, cloud-optional" — the MCP server runs on localhost alongside the existing daemon.

---

2. Target Users

Primary: Multi-runtime developers

Developers using Claude Code alongside other agent runtimes (n8n, LangGraph, custom Python agents) who want a single place to see what all their agents did. They already use AR for Claude Code observability and want to extend it.

Evidence of demand: The r/mcp Reddit thread on AR's token monitoring post drew engagement specifically around subagent-to-parent cost roll-up and cross-runtime observability. User "IllegitimateGoat" explicitly praised the ability to roll up costs from sub agents to their parent agent — a feature that requires programmatic access to the event tree.

Secondary: Dashboard and monitoring builders

Developers building custom dashboards, Grafana panels, or alerting pipelines who need structured observability data from agent sessions. Today they'd have to query SQLite directly. An MCP interface gives them a stable, versioned contract.

Tertiary: Agent self-monitoring

Agents that want to introspect their own execution — checking token budget remaining, querying their own history, or deciding whether to continue a task based on cost so far. The MCP server makes AR's data available as tools an agent can call mid-session.

---

3. Goals and Non-Goals

Goals

1. Expose AR's recorded session and event data through standard MCP tools (read path).
2. Accept execution events from external agent runtimes through standard MCP tools (write path).
3. Expose token consumption and cost estimation data through MCP tools.
4. Maintain AR's privacy guarantees — redaction, truncation, no prompt capture — for all data served through MCP.
5. Follow the existing AR architecture: new `packages/mcp-server` package, Fastify-served, localhost-only, fail-open.
6. Support both Streamable HTTP and STDIO transports so the MCP server works with any client.
7. Ensure the MCP server is consumable by any MCP-capable runtime, not coupled to any specific framework.

Non-Goals

- Cloud sync or remote access. The MCP server is localhost-only, consistent with AR v1 scope.
- Real-time streaming of events via MCP subscriptions. Polling or request/response only in v1.
- Dollar cost calculation with live provider pricing. Token counts are first-class; dollar estimates use a static, user-configurable pricing table.
- Authentication or multi-user access control. Localhost-only implies single-user trust boundary.
- Replacing the existing REST API. The MCP server is an additional interface, not a replacement.
- Prompt or chain-of-thought capture through the write path. The same redaction rules apply.

---

4. Feature Specification

4.1 Phase 1 — Read Path (MCP Tools for Querying)

The read path exposes AR's existing data through MCP tools. No new data is created; these tools query SQLite and return structured results.

Tool: `ar_list_sessions`

Purpose: List recorded sessions with summary metrics.

Parameters:

Name	Type	Required	Description
`limit`	integer	No	Max sessions to return. Default: 20. Max: 100.
`offset`	integer	No	Pagination offset. Default: 0.
`status`	string	No	Filter by status: `active`, `completed`, `error`, `all`. Default: `all`.
`since`	string (ISO 8601)	No	Only sessions started after this timestamp.
`upstream_key`	string	No	Filter by upstream MCP server key.

Returns: Array of session summaries:

```
{
  "sessions": [
    {
      "session_id": "abc-123",
      "started_at": "2026-03-26T10:00:00Z",
      "ended_at": "2026-03-26T10:05:32Z",
      "status": "completed",
      "event_count": 47,
      "total_input_tokens": 12840,
      "total_output_tokens": 3210,
      "total_tokens": 16050,
      "context_budget_percent": 10.7,
      "error_count": 1,
      "upstream_keys": ["linear", "github", "filesystem"]
    }
  ],
  "total": 142,
  "limit": 20,
  "offset": 0
}
```

Tool: `ar_get_session`

Purpose: Return the full event tree for a session, with token counts rolled up from children to parents.

Parameters:

Name	Type	Required	Description
`session_id`	string	Yes	The session to retrieve.
`depth`	integer	No	Max tree depth to return. Default: unlimited.
`event_types`	string[]	No	Filter by event type: `agent_call`, `subagent_call`, `skill_call`, `tool_call`. Default: all.
`include_io`	boolean	No	Include redacted/truncated input and output payloads. Default: false.

Returns: Hierarchical event tree:


```
{
  "session_id": "abc-123",
  "started_at": "2026-03-26T10:00:00Z",
  "status": "completed",
  "total_tokens": 16050,
  "estimated_cost_usd": 0.0482,
  "events": [
    {
      "event_id": "evt-001",
      "event_type": "agent_call",
      "parent_event_id": null,
      "started_at": "2026-03-26T10:00:00Z",
      "ended_at": "2026-03-26T10:05:32Z",
      "status": "success",
      "tokens": {
        "self_input": 500,
        "self_output": 120,
        "children_input": 12340,
        "children_output": 3090,
        "total": 16050
      },
      "children": [
        {
          "event_id": "evt-002",
          "event_type": "tool_call",
          "parent_event_id": "evt-001",
```

```
          "tool_name": "linear_search_issues",
          "upstream_key": "linear",
          "started_at": "2026-03-26T10:00:01Z",
          "duration_ms": 340,
          "status": "success",
          "tokens": {
            "self_input": 890,
            "self_output": 2100,
            "children_input": 0,
            "children_output": 0,
            "total": 2990
          },
          "children": []
        }
      ]
    }
  ]
}
```

Token roll-up logic: Each event's `tokens.total` equals `self_input + self_output + children_input + children_output`. Children totals are the recursive sum of all descendant events. This is computed at query time from the stored per-event token counts, not stored redundantly.

Tool: **`ar_query_token_usage`**

Purpose: Query aggregated token usage across sessions, with filtering and grouping.

Parameters:

Name	Type	Required	Description
`group_by`	string	No	Grouping dimension: `session`, `upstream_key`, `tool_name`, `event_type`, `hour`, `day`. Default: `session`.
`since`	string (ISO 8601)	No	Start of time range.
`until`	string (ISO 8601)	No	End of time range.
`upstream_key`	string	No	Filter to a specific upstream.

`session_id`	string	No	Filter to a specific session.
`limit`	integer	No	Max results. Default: 50.

Returns: Aggregated token usage:


```
{
  "group_by": "upstream_key",
  "period": {
    "since": "2026-03-25T00:00:00Z",
    "until": "2026-03-26T23:59:59Z"
  },
  "groups": [
    {
      "key": "linear",
      "call_count": 89,
      "total_input_tokens": 45200,
      "total_output_tokens": 128900,
      "total_tokens": 174100,
      "estimated_cost_usd": 0.52,
      "error_count": 3,
      "avg_duration_ms": 420
    },
    {
      "key": "github",
      "call_count": 34,
      "total_input_tokens": 12100,
      "total_output_tokens": 8900,
      "total_tokens": 21000,
      "estimated_cost_usd": 0.063,
      "error_count": 0,
      "avg_duration_ms": 280
    }
  ],
  "totals": {
    "call_count": 123,
    "total_tokens": 195100,
    "estimated_cost_usd": 0.583
  }
}
```


Tool: **`ar_get_token_budget`**

Purpose: Return the current token budget status for an active session.

Parameters:

Name	Type	Required	Description
`session_id`	string	Yes	The session to check.

Returns:


```
{
  "session_id": "abc-123",
  "budget_tokens": 150000,
  "used_tokens": 16050,
  "remaining_tokens": 133950,
  "percent_used": 10.7,
  "threshold_warning": false,
  "threshold_percent": 75,
  "status": "active"
}
```


Use case: An agent calls this mid-session to decide whether to continue a complex task or stop. A monitoring tool polls this to trigger alerts.

Tool: `ar_list_upstreams`

Purpose: List all known upstream MCP servers with their activity metrics.

Parameters:

Name	Type	Required	Description
`since`	string (ISO 8601)	No	Only include activity since this time.

Returns:


```
{
  "upstreams": [
    {
      "upstream_key": "linear",
      "last_seen": "2026-03-26T10:05:30Z",
      "total_calls": 892,
      "total_tokens": 1240000,
      "error_rate_percent": 2.1,
      "tools_used": ["linear_search_issues", "linear_create_issue", "linear_updat
    }
```

```
    ]
  }
```

4.2 Phase 2 — Write Path (MCP Tools for Event Ingestion)

The write path allows external agent runtimes to push their own execution events into AR's timeline. This is the "multi-project" play — any MCP client can record to AR without being proxied by it.

Design Principles for the Write Path

1. **Same event model.** External events use the same `event_id`, `parent_event_id`, `event_type`, timestamp, and token fields as proxy-captured events. No separate schema.
2. **Source attribution.** Every externally ingested event carries a `source` field (e.g., `"n8n"`, `"langgraph"`, `"syrin"`, `"custom"`) that distinguishes it from proxy-captured events (`source: "proxy"`). The UI and CLI can filter or label by source.
3. **Same redaction rules.** Input and output payloads submitted through the write path are subject to the same `AR_REDACT_KEYS` redaction and truncation limits as proxy-captured data. The write path never stores raw secrets even if a client sends them.
4. **Fail-open on ingestion.** If event storage fails (e.g., SQLite lock contention), the MCP tool returns an error response but the calling agent is not blocked. Consistent with AR's proxy behavior.
5. **No prompt capture.** The write path accepts tool inputs/outputs and metadata. It does not accept, store, or acknowledge prompt text or chain-of-thought content. If a client sends a field named `prompt`, `reasoning`, or `chain_of_thought`, it is silently dropped.
6. **Idempotent writes.** If a client sends an event with an `event_id` that already exists, the write is ignored (not rejected, not updated). This allows clients to retry safely.

Tool: **`ar_record_event`**

Purpose: Record a single execution event into AR's timeline.

Parameters:

Name	Type	Required	Description
`event_type`	string	Yes	One of: `agent_call`, `subagent_call`, `skill_call`, `tool_call`.

`event_id`	string	No	Client-generated unique ID. If omitted, AR generates one.
`parent_event_id`	string	No	ID of the parent event. Null for top-level events.
`session_id`	string	Yes	Session this event belongs to. Created automatically if new.
`source`	string	Yes	Identifier for the calling runtime (e.g., `"n8n"`, `"langgraph"`, `"custom"`).
`tool_name`	string	No	For `tool_call` events: the tool that was called.
`upstream_key`	string	No	For `tool_call` events: which MCP server the tool belongs to.
`started_at`	string (ISO 8601)	Yes	When the event started.
`ended_at`	string (ISO 8601)	No	When the event ended. Null if still in progress.
`status`	string	No	One of: `started`, `success`, `error`, `timeout`. Default: `started`.
`error_category`	string	No	Stable error category if status is `error` (e.g., `timeout`, `auth_failure`, `validation_error`, `upstream_error`, `unknown`).
`error_message`	string	No	Human-readable error message. Truncated to 500 chars.
`input_tokens`	integer	No	Estimated input token count for this event.
`output_tokens`	integer	No	Estimated output token count for this event.
`input_preview`	string	No	Redacted/truncated preview of the input. Max 2000 chars. Subject to `AR_REDACT_KEYS`.
`output_preview`	string	No	Redacted/truncated preview of the output. Max 2000 chars. Subject to

			`AR_REDACT_KEYS`.
`metadata`	object	No	Arbitrary key-value metadata (max 10 keys, 500 chars per value). Subject to redaction.

Returns:

```
{
  "event_id": "evt-ext-001",
  "session_id": "ext-session-42",
  "stored": true,
  "deduplicated": false
}
```

Validation rules:

- `event_type` must be one of the four allowed values. Unknown types are rejected.
- `session_id` is required. If the session doesn't exist, it is created with `source` as its origin.
- `started_at` is required and must be a valid ISO 8601 timestamp.
- `parent_event_id`, if provided, must reference an existing event in the same session. If the parent doesn't exist yet (out-of-order ingestion), the event is stored with the parent reference and resolved when the parent arrives.
- `input_preview` and `output_preview` are truncated at 2000 characters after redaction.
- `metadata` is limited to 10 keys with values of 500 characters each, after redaction.
- Fields named `prompt`, `reasoning`, `chain_of_thought`, `system_prompt`, or `messages` are silently dropped from `metadata`.

Tool: `ar_complete_event`

Purpose: Update an in-progress event with completion data. This supports the pattern where an agent records `ar_record_event` with `status: "started"` at the beginning of a task, then calls `ar_complete_event` when it finishes.

Parameters:

Name	Type	Required	Description
`event_id`	string	Yes	The event to complete.

`session_id`	string	Yes	The session the event belongs to.
`ended_at`	string (ISO 8601)	Yes	When the event ended.
`status`	string	Yes	Final status: `success`, `error`, `timeout`.
`error_category`	string	No	If status is `error`.
`error_message`	string	No	If status is `error`. Truncated to 500 chars.
`input_tokens`	integer	No	Final input token count. Overwrites previous value.
`output_tokens`	integer	No	Final output token count. Overwrites previous value.
`output_preview`	string	No	Redacted/truncated output preview. Max 2000 chars.

Returns:

```
{
  "event_id": "evt-ext-001",
  "updated": true
}
```

Validation: The event must exist, must belong to the specified session, and must have `status: "started"`. Completing an already-completed event is a no-op (returns `updated: false`).

Tool: **`ar_record_batch`**

Purpose: Record multiple events in a single call. Designed for runtimes that batch their execution logs (e.g., an n8n workflow that completes and reports all steps at once).

Parameters:

Name	Type	Required	Description
`session_id`	string	Yes	Session all events belong to.

`source`	string	Yes	Identifier for the calling runtime.
`events`	array	Yes	Array of event objects, each following the `ar_record_event` schema (excluding `session_id` and `source`, which are set at the batch level). Max 100 events per batch.

Returns:

```
{
  "session_id": "ext-session-42",
  "stored": 47,
  "deduplicated": 3,
  "errors": []
}
```

Ordering: Events within a batch are processed in array order. Parent references within the same batch are resolved positionally — a child can reference a parent that appears earlier in the same batch by `event_id`.

4.3 Cost Estimation Layer

Dollar cost estimation is a separate, configurable concern layered on top of token counts. It is not part of the MCP tools' core contract — tools always return token counts as first-class data, and cost estimates are clearly labeled as estimates.

Pricing Configuration

Pricing data is stored in a JSON configuration file at `AR_PRICING_PATH` (default: `.storage/pricing.json`):

```
{
  "version": 1,
  "updated_at": "2026-03-26T00:00:00Z",
  "providers": {
    "anthropic": {
      "claude-sonnet-4": {
        "input_per_million": 3.00,
        "output_per_million": 15.00
      },
      "claude-haiku-4": {
        "input_per_million": 0.80,
        "output_per_million": 4.00
      }
```

```
    },
    "openai": {
      "gpt-4o": {
        "input_per_million": 2.50,
        "output_per_million": 10.00
      }
    },
    "default": {
      "input_per_million": 3.00,
      "output_per_million": 15.00
    }
  }
}
```

Design decisions:

- **Static file, user-editable.** AR ships with a default pricing file. Users can update it. AR does not fetch pricing from APIs.
- **Default fallback.** If a model isn't in the pricing file, the `default` rates are used. This ensures cost estimates are always available, even if imprecise.
- **No tiered pricing in v1.** Cache read/write costs, batch discounts, and tiered input pricing are explicitly out of scope. The complexity of tracking these accurately across providers is prohibitive for a local tool. The pricing file can be extended in v2 if demand warrants it.
- **Labeled as estimates.** All cost fields in MCP tool responses use the prefix `estimated_` (e.g., `estimated_cost_usd`) to signal that these are approximations, not billing-grade numbers.
- **Model attribution.** For proxy-captured events, the model is not directly observable (it's in the prompt, which AR doesn't capture). Cost estimates for proxy events use the `default` rate. For externally ingested events, clients can include a `model` field in `metadata` which AR uses for pricing lookup.

---

5. Architecture

5.1 Package Structure

The MCP server is implemented as a new package: `packages/mcp-server`.

```
packages/mcp-server/
```

```
├── src/
│   ├── index.ts              # Package entry point
│   ├── server.ts             # MCP server setup (tool registration, transport)
│   ├── tools/
│   │   ├── read/
│   │   │   ├── list-sessions.ts
│   │   │   ├── get-session.ts
│   │   │   ├── query-token-usage.ts
│   │   │   ├── get-token-budget.ts
│   │   │   └── list-upstreams.ts
│   │   └── write/
│   │       ├── record-event.ts
│   │       ├── complete-event.ts
│   │       └── record-batch.ts
│   ├── pricing/
│   │   ├── estimator.ts       # Token-to-cost calculation
│   │   └── config.ts          # Pricing file loader
│   ├── rollup/
│   │   └── token-rollup.ts    # Recursive tree aggregation
│   └── validation/
│       ├── schemas.ts          # Input validation schemas (Zod)
│       └── redaction.ts        # Re-exports from @agent-recorder/core
├── tests/
│   ├── read/
│   └── write/
└── package.json
```

5.2 Integration with Existing Daemon

The MCP server is mounted as an additional route on the existing Fastify daemon in `packages/service`:

```
┌─────────────────────────────────────────────┐
│  Fastify Daemon (packages/service)          │
│                                             │
│  /api/*          → REST API (existing)      │
│  /proxy/*        → MCP Proxy (existing)     │
│  /mcp            → MCP Server (new)         │
│  /sse            → SSE transport (new)      │
│                                             │
│  All routes share the same SQLite instance  │
└─────────────────────────────────────────────┘
```

The MCP server uses the same SQLite database and the same `@agent-recorder/core` storage layer as the proxy and REST API. No data duplication. Read tools execute queries;

write tools insert rows using the existing event storage functions.

5.3 Transports

The MCP server supports two transports:

Streamable HTTP (primary): Served at `http://localhost:{AR_LISTEN_PORT}/mcp`. This is the modern MCP transport and works with any HTTP-capable MCP client.

STDIO (secondary): A separate entry point (`packages/mcp-server/src/stdio.ts`) that wraps the same tool implementations in STDIO transport. This allows the MCP server to be configured in `claude_desktop_config.json` or `.vscode/mcp.json` as a spawnable process:

```
{
  "mcpServers": {
    "agent-recorder": {
      "command": "pnpm",
      "args": ["ar", "mcp-server", "--stdio"]
    }
  }
}
```

5.4 Data Flow

```
Read Path:
  MCP Client → tools/call → ar_list_sessions
                          → SQLite query
                          → token roll-up computation
                          → cost estimation
                          → MCP response

Write Path:
  MCP Client → tools/call → ar_record_event
                          → input validation (Zod)
                          → redaction (AR_REDACT_KEYS)
                          → prompt field stripping
                          → truncation
                          → SQLite insert (existing storage layer)
                          → MCP response

Proxy-Captured Events (existing, unchanged):
  Claude Code → MCP Proxy → downstream MCP server
                    ↓
              SQLite insert (source: "proxy")
```

Both proxy-captured and externally-ingested events land in the same SQLite tables, queryable through the same read tools. The `source` field distinguishes them.

---

6. Privacy and Security

6.1 Redaction

All data served through read tools and accepted through write tools passes through the existing redaction pipeline:

- `AR_REDACT_KEYS` is applied to `input_preview`, `output_preview`, and `metadata` values.
- Token counts, timestamps, event IDs, and status fields are never redacted (they contain no sensitive content).
- The `include_io` parameter on `ar_get_session` defaults to `false`, so casual queries never expose payloads.

6.2 Prompt Stripping

The write path silently drops fields that could contain prompts or reasoning:

- `prompt`
- `system_prompt`
- `reasoning`
- `chain_of_thought`
- `messages`
- `thought`
- `thinking`

These are dropped from `metadata` and from the top-level event payload. This is enforced at the validation layer before storage.

6.3 Localhost-Only

The MCP server is bound to `127.0.0.1` only. It is not accessible from other machines on the network. This is consistent with AR's existing daemon behavior and the "local-first" principle.

6.4 Rate Limiting

To prevent accidental abuse (e.g., a runaway agent flooding the write path):

- `ar_record_event` : Max 100 calls per second per session.
- `ar_record_batch` : Max 10 calls per second, max 100 events per batch.
- `ar_complete_event` : Max 100 calls per second per session.
- Read tools: Max 50 calls per second.

Exceeding limits returns a standard MCP error with a `Retry-After` hint. Rate limiting never affects the MCP proxy path — consistent with the "fail open, never block proxying" principle.

---

7. CLI Integration

New CLI commands for the MCP server:

```
pnpm ar mcp-server start          # Start standalone MCP server (if daemon is not
pnpm ar mcp-server status         # Check if MCP server is active
pnpm ar mcp-server --stdio        # Start in STDIO mode (for client configs)
```

When the daemon is running (`pnpm ar start`), the MCP server is automatically available at `/mcp` . No separate start command needed.

---

8. Phasing and Milestones

Phase 1: Read Path (Target: v2.1.0)

Scope: All five read tools (`ar_list_sessions` , `ar_get_session` , `ar_query_token_usage` , `ar_get_token_budget` , `ar_list_upstreams`) with Streamable HTTP transport.

Deliverables:

- `packages/mcp-server` package with read tools
- Token roll-up computation (recursive tree aggregation from existing per-event data)
- Static pricing configuration and cost estimation
- Mounted on existing Fastify daemon at `/mcp`

- Integration tests with MCP Inspector
- Documentation in `docs/mcp-server.md`

Success criteria:

- An MCP client (e.g., MCP Inspector, Claude Desktop) can connect to `http://localhost:8787/mcp` and call all five read tools.
- `ar_get_session` returns a correctly aggregated token tree where parent totals equal the sum of their children.
- Cost estimates are present and use the static pricing file.

Phase 2: Write Path (Target: v2.2.0)

Scope: All three write tools (`ar_record_event` , `ar_complete_event` , `ar_record_batch`) with full validation, redaction, and prompt stripping.

Deliverables:

- Write tools in `packages/mcp-server`
- Input validation schemas (Zod)
- Prompt field stripping
- Redaction integration for write path
- Idempotent write handling
- Rate limiting
- STDIO transport support
- Integration tests with a mock external agent recording events

Success criteria:

- An external agent (e.g., a Python script using the MCP SDK) can record events to AR and see them in the CLI (`pnpm ar sessions view`) alongside proxy-captured events.
- Events from different sources are visually distinguishable in the UI and CLI.
- Prompt fields sent through the write path are not stored.
- Duplicate `event_id` submissions are silently deduplicated.

Phase 3: Ecosystem Validation (Target: v2.3.0)

Scope: Integration guides and tested examples for at least two external runtimes.

Deliverables:

- Integration guide: "Recording n8n workflow events in Agent Recorder"
- Integration guide: "Recording LangGraph agent events in Agent Recorder"
- Example: Python script using `@modelcontextprotocol/sdk` to record events
- Example: Claude Code session querying its own token budget via the MCP server

Success criteria:

- A developer can follow the n8n guide and see n8n workflow events in AR's timeline within 15 minutes.
- The AR UI shows a unified timeline mixing proxy-captured Claude Code events and externally-ingested n8n events, with correct parent/child relationships within each source.

---

9. Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
Write path abuse — a runaway agent floods SQLite with events	Medium	Medium	Rate limiting per session. SQLite WAL mode handles concurrent writes. Fail-open: if storage fails, return error but don't crash.
Schema instability — changing the MCP tool schemas breaks external clients	Medium	High	Version the tool names (e.g., `ar_list_sessions` not `list_sessions`). Add a `schema_version` field to responses. Commit to backwards compatibility within a major version.
Token roll-up performance — deep event trees cause slow recursive queries	Low	Medium	Limit default tree depth to 10. Add `depth` parameter. Consider materialized roll-up columns if profiling shows issues (optimization, not v1 requirement).
Pricing data staleness —			Label all costs as "estimated." Ship updated pricing files with AR

default pricing file becomes inaccurate as providers change rates	High	Low	releases. Allow user overrides. Document that pricing is approximate by design.
Scope creep toward real-time — users want event streaming, WebSocket subscriptions	Medium	Medium	Explicitly non-goal in v1. Document the polling pattern. Evaluate SSE-based subscriptions for v3 if demand is validated.
External event quality — external runtimes send malformed or meaningless events	Medium	Low	Strict Zod validation. Reject invalid events with clear error messages. Truncate oversized fields. Don't attempt to fix bad data.

---

10. Open Questions

1. **Should the MCP server be enabled by default when the daemon starts, or opt-in via configuration?** Recommendation: enabled by default, since it's localhost-only and adds no security surface beyond what the REST API already exposes.
2. **Should externally-ingested sessions be visually separated from proxy-captured sessions in the UI, or merged into a single timeline?** Recommendation: merged, with a `source` badge on each event. Users who want separation can filter by source.
3. **Should the write path accept custom `event_type` values beyond the four defined types, to accommodate runtimes with different execution models?** Recommendation: no in v1. Stick with the four types. If a runtime has a concept that doesn't map (e.g., "workflow_step"), it should use the closest match (`tool_call`) with a descriptive `metadata` field. Custom types can be added in v2 if a clear pattern emerges.
4. **Should `ar_record_event` accept a `model` field at the top level (not just in metadata) to enable accurate cost estimation?** Recommendation: yes, add it as an optional top-level field. This is the one piece of information that meaningfully improves cost estimates without introducing prompt-capture risk.

---

11. Success Metrics

- **Adoption (read):** At least 3 unique MCP clients successfully querying AR within 30 days of Phase 1 launch (tracked via MCP tool call logs).

- **Adoption (write):** At least 1 external runtime successfully recording events within 30 days of Phase 2 launch.
- **Data quality:** Zero instances of prompt or chain-of-thought content found in stored external events (audited via periodic grep of the `input_preview` and `output_preview` columns).
- **Reliability:** MCP server uptime matches daemon uptime (no independent crashes). Zero instances of the MCP server blocking the proxy path.
- **Performance:** Read tools respond in under 200ms for sessions with up to 500 events. Write tools respond in under 50ms per event.

---

12. References

- Agent Recorder v2.0.14 release: Token Monitoring — Discussion #27
- MCP Specification — modelcontextprotocol.io
- Agent Recorder Architecture — `docs/architecture.md`
- Agent Recorder Product Principles — `docs/product-principles.md`
- Existing feature request — `docs/feature-request-mcp-server.md`
- r/mcp user feedback on token roll-up and cross-runtime observability